INSTITUTO PROFESIONAL SAN SEBASTIAN

INSTITUTO PROFESIONAL ACREDITADO
NIVEL AVANZADO
CNA
DOCENCIA Y RESULTADOS DEL PROCESO DE FORMACIÓN
GESTIÓN ESTRATÉGICA Y RECURSOS INSTITUCIONALES
ASEGURAMIENTO INTERNO DE LA CALIDAD
HASTA SEPTIEMBRE 2029

Evaluación sumativa N° 1

Docente: Boris Belmar

Asignatura: Desarrollo de aplicaciones móviles

Sección: 51

Integrantes: Florencia Vergara - Agustina Barbaroy - Andrea Carreño

1. Introducción

Este informe presenta el desarrollo de “Mis Tareas”, una aplicación móvil para registrar, consultar, editar y eliminar tareas pendientes, cada una con su título y descripción. Buscamos resolver una necesidad cotidiana: tener claro lo que hay por hacer, con una interfaz sencilla y agradable de usar.

Aunque la pauta propone Ionic, desarrollamos el proyecto con **React Native y Expo**, con la autorización del docente. Ambas opciones permiten construir una aplicación móvil con componentes reutilizables y React. En la sección 3.4 mostramos cómo se corresponden los componentes solicitados con los que utilizamos.

2. Análisis del problema

2.1. Requisitos

Requisitos	Cómo se cumple
Página principal con lista de tareas	Pantalla “Mis tareas”, con FlatList
Cada tarea muestra título y descripción	Componente TaskItem
Formulario para agregar tareas	Pantalla task-form con dos campos TextField
Botón para guardar	Botón “Guardar tarea” con icono
El formulario limpia sus campos al agregar	Se reinicia el estado tras guardar; “Guardar y agregar otra” lo deja a la vista
Editar tareas	Botón de lápiz, o tocar la tarea
Botón de eliminar junto a cada tarea	Icono de papelera, con confirmación

2.2. Funciones adicionales y su motivo

- **Marcar como completada:** es la acción que más se repite al usar una lista de tareas.
- **Filtros y orden:** ayudan a encontrar lo que se necesita cuando la lista crece.
- **Resumen de avance:** permite ver cuánto se ha hecho y cuánto falta.

- **Almacenamiento local:** las tareas se conservan al cerrar la aplicación. La pauta lo contempla para unidades posteriores, pero decidimos adelantarlo porque perder la información afectaría seriamente la experiencia de uso.
- **Temas:** se puede elegir entre Claro, Oscuro, Rosado, Azul y Verde, o dejar el modo automático, que sigue la configuración del teléfono. La elección queda guardada.

3. Conceptos fundamentales del framework

3.1. Qué son React, React Native y Expo

React es una biblioteca para construir interfaces a partir de componentes. React Native lleva ese modelo a los teléfonos: escribimos el código en TypeScript, pero en pantalla se muestran vistas nativas de Android e iOS, no una página web dentro de la aplicación. Expo es el framework que se apoya en React Native y suma herramientas, módulos listos para usar, navegación y servicios de compilación.

Node.js permite ejecutar JavaScript fuera del navegador y es la base de nuestras herramientas de desarrollo: con él funcionan el servidor de Expo y npm, el gestor con el que instalamos las dependencias registradas en package.json. Se utiliza solo durante el desarrollo; no forma parte de la aplicación instalada en el teléfono.

3.2. Arquitectura

La aplicación funciona en tres capas:

- **JavaScript:** el código de la aplicación se ejecuta en el motor **Hermes**, optimizado para móviles. Metro lo empaqueta.
- **Nueva Arquitectura de React Native: JSI** conecta JavaScript con el código nativo sin serializar mensajes, **Fabric** renderiza la interfaz y los **Turbo Modules** cargan los módulos nativos.
- **Plataforma nativa:** Android e iOS dibujan los componentes. Por ejemplo, Pressable y TextInput se convierten en vistas propias de cada sistema.

Además, el proyecto usa **Continuous Native Generation**: las carpetas android e ios no se editan a mano, sino que se generan desde app.json. Allí se configuran el nombre, los iconos, la pantalla de inicio y los identificadores de la aplicación.

3.3. Terminología y aplicación en el proyecto

Término	Significado	Ejemplo en el proyecto
Componente	Pieza reutilizable de interfaz	Button, TaskItem
Props	Datos que un componente recibe	<Icon name="edit" />
Estado	Datos que cambian y actualizan la interfaz	Filtro seleccionado
Hook	Función que da acceso a funciones de React	useState, useTasks
Context	Datos compartidos sin pasar props	TasksProvider
Reducer	Función que calcula el siguiente estado	tasksReducer
Metro	Empaquetador del código	npx expo start
Expo Router	Navegación basada en archivos	Carpeta src/app
Layout	Archivo que define un navegador	_layout.tsx
Safe area	Zona no cubierta por muescas o barras	useSafeAreaInsets
React Compiler	Optimiza renderizados automáticamente	Activo en app.json

3.4. Correspondencia con los componentes de Ionic

Ionic (pauta)	React Native y Expo (proyecto)
ion-header	ScreenHeader
ion-content	ScrollView y FlatList
ion-list	FlatList, con virtualización de elementos
ion-item	TaskItem
ion-input	TextField, basado en TextInput
ion-button	Button, basado en Pressable

Ionic (pauta)	React Native y Expo (proyecto)
ionic serve (puerto 8100)	npm run web (también en el puerto 8100)

4. Patrones de diseño

En esta sección explicamos los patrones centrales del framework y cómo los aplicamos en el código de la aplicación.

4.1. Composición de componentes

En lugar de heredar de clases, las interfaces se construyen combinando piezas pequeñas. Por ejemplo, ScreenHeader recibe acciones como props y contenido como children, lo que nos permitió usar el mismo encabezado en todas las pantallas.

```
<ScreenHeader
  title="Mis tareas"
  subtitle={subtitle}
  trailing={<IconButton icon="more" label="Más opciones" ... />}>
  <ProgressBar value={stats.progress} ... />
</ScreenHeader>
```

4.2. Flujo de datos unidireccional y reducer

El estado fluye de los componentes superiores hacia los inferiores, y los cambios suben como acciones. Todas las modificaciones de las tareas pasan por una sola función pura, lo que hace que cada cambio sea predecible. Además, el tipo TasksAction permite que TypeScript detecte acciones inválidas.

```
function tasksReducer(tasks: Task[], action: TasksAction): Task[] {
  switch (action.type) {
    case 'add':
      return [action.task, ...tasks];
    case 'remove':
      return tasks.filter((task) => task.id !== action.id);
    // ...
  }
}
```

4.3. Provider con Context

TasksProvider pone las tareas a disposición de cualquier pantalla, sin tener que pasarlas por componentes intermedios. Gracias a esto, la insignia de la barra de pestañas y la lista siempre muestran el mismo dato.

```
<AppThemeProvider>
  <FeedbackProvider>
    <TasksProvider>
      <RootNavigator />
    </TasksProvider>
  </FeedbackProvider>
</AppThemeProvider>
```

AppThemeProvider aplica el mismo patrón a la apariencia: al elegir un tema en Ajustes, toda la aplicación cambia de inmediato, porque cada componente obtiene sus colores mediante el hook useTheme.

4.4. Hooks personalizados

Los hooks permiten reutilizar lógica entre componentes. useTasks, además, previene un uso incorrecto: si un componente queda fuera del Provider, muestra un error claro.

```
export function useTasks() {
  const context = use(TasksContext);
  if (!context) throw new Error('useTasks debe usarse dentro de TasksProvider');
  return context;
}
```

4.5. Componentes controlados

En el formulario, el valor de cada campo vive en el estado de React. Esto nos permite validar los datos, mostrar el contador de caracteres, detectar cambios sin guardar y limpiar los campos después de agregar una tarea.

```
<TextField
  value={draft.title}
  error={errors.title}
  onChangeText={(value) => updateField('title', value)}
/>
```

4.6. Capa de servicio

Las pantallas no necesitan saber cómo se guardan los datos: de eso se encargan task-storage.ts y theme-storage.ts, mediante AsyncStorage. Así, si en una unidad posterior las tareas se guardan en una API, bastará con modificar esta capa. Además, los datos se validan al leerlos, para que información dañada no bloquee la aplicación.

```
export async function loadTasks(): Promise<Task[]> {
  const stored = await AsyncStorage.getItem(STORAGE_KEY);
  if (!stored) return [];
  const parsed: unknown = JSON.parse(stored);
  return Array.isArray(parsed) ? parsed.filter(isTask) : [];
}
```

4.7. Adaptador por plataforma

El componente Icon oculta las diferencias entre sistemas: usa SF Symbols en iOS y Material Symbols en Android, de modo que cada icono se ve propio de su plataforma, mientras que el resto del código solo indica un nombre.

```
const ICONS = {
  delete: { ios: 'trash', android: 'delete', web: 'delete' },
  // ...
};
```

Expo también aplica este patrón al resolver archivos: use-color-scheme.web.ts reemplaza automáticamente a use-color-scheme.ts en web.

4.8. Enrutamiento basado en archivos y navegadores anidados

Expo Router convierte cada archivo de src/app en una ruta. Usamos una pila (Stack) que contiene las pestañas (Tabs) y el formulario, el cual recibe un parámetro para distinguir si se está creando o editando una tarea.

```
router.push('/task-form');                          // crear
router.push({ pathname: '/task-form', params: { id } });    // editar
```

4.9. Casos de uso avanzados

- **Observador:** useEffect suscribe el botón físico “atrás” de Android mientras haya cambios sin guardar y cancela la suscripción al terminar.
- **Render props:** FlatList recibe renderItem y las pestañas reciben tabBarIcon, funciones que indican cómo dibujar cada elemento.
- **Reinicio por clave:** <TaskForm key={id ?? 'new'} /> crea un formulario nuevo para cada tarea, sin arrastrar datos de la anterior.
- **Carga asíncrona con estado de espera:** la pantalla de inicio se mantiene hasta leer las tareas y el tema, lo que evita mostrar una lista vacía por un instante.

5. Diseño de la interfaz

5.1. Principios aplicados

Principio	Aplicación
Jerarquía visual	Escala tipográfica de seis niveles (de 30 a 13 px). El encabezado con degradado y el botón “Nueva tarea” destacan sobre el resto.
Balance	Tres filtros de igual ancho, tres tarjetas de igual tamaño en Resumen y márgenes simétricos de 16 px.
Contraste	Se midieron 19 combinaciones en cada uno de los cinco temas: todo el texto supera 4,5:1 (WCAG AA), e iconos, bordes de campo y barra de progreso superan 3:1.
Ritmo	Todo el espaciado usa una misma escala (2, 4, 8, 16, 24, 32 y 64 px) y las tarjetas se repiten con igual estructura.
Proporción	Áreas táctiles de 48 px, iconos de 16 a 24 px en los controles y un ancho máximo de contenido de 640 px en pantallas grandes.

5.2. Temas y colores

La aplicación ofrece cinco temas de tonos sobrios, que se eligen en Ajustes, además de un modo automático. Definimos los colores como tokens en src/constants/themes.ts, nunca como valores sueltos en las pantallas. El rojo se reserva para errores y acciones destructivas, y siempre va acompañado de un icono o texto.

Tema	Estilo	Primario	Encabezado	Fondo
Claro	Marfil y grafito	#2A2824	#1F1E1B	#F8F6F1
Oscuro	Grafito y champán	#E6DAC0	#0F0F11	#131315
Rosado	Rosa antiguo	#8E2A48	#5C1A30	#FBF5F5
Azul	Azul medianoche	#1B4F8F	#0F2A4D	#F4F7FB
Verde	Verde bosque	#1B6B4E	#0F3B2B	#F3F7F4

5.3. Transparencias

Las tarjetas son translúcidas y dejan ver un suave degradado de fondo. Los filtros, botones secundarios y bordes también usan el color del tema con baja opacidad. Optamos por transparencias sin desenfoque porque su contraste se puede calcular y su resultado es más predecible entre dispositivos. Los menús, diálogos y campos de texto son opacos, ya que su contenido siempre debe leerse con claridad.

5.4. Contraste medido

Para asegurar que los colores fueran legibles en todos los temas, usamos la librería culori, que calcula el contraste según WCAG. Como las tarjetas son translúcidas, cada color se mezcla primero con su fondo y se considera el punto más desfavorable del degradado. La revisión se ejecuta con npm run check:contrast y termina con error si alguna combinación no cumple, lo que nos permite verificar el contraste cada vez que se modifica un color. Se revisaron 19 combinaciones en cada uno de los cinco temas, 95 en total. La revisión detectó que el borde de los campos no llegaba a 3:1 en los temas Rosado, Azul y Verde; tras corregirlo, todas cumplen. Las Figuras 1 y 2 muestran el resultado de los temas Verde y Claro, y la siguiente tabla resume los valores más bajos de cada tema.

Tema	Texto (mínimo 4,5:1)	Borde de campo (mínimo 3:1)	Progreso (mínimo 3:1)
Claro	5,03:1	3,16:1	4,52:1
Oscuro	6,59:1	4,42:1	5,54:1
Rosado	5,51:1	3,40:1	3,67:1
Azul	5,25:1	3,32:1	3,56:1
Verde	5,09:1	3,43:1	3,15:1

Tema: green (menor contraste de texto: 5.09:1, Subtítulo del encabezado)

(index)	pair	minimum	worst	ok
0	'Texto sobre fondo'	4.5	14.13	'sí'
1	'Texto sobre tarjeta'	4.5	15.98	'sí'
2	'Texto secundario sobre fondo'	4.5	6.06	'sí'
3	'Texto secundario sobre tarjeta'	4.5	6.86	'sí'
4	'Texto secundario sobre superficie'	4.5	7.18	'sí'
5	'Filtro sin seleccionar'	4.5	5.39	'sí'
6	'Filtro seleccionado y botón'	4.5	6.44	'sí'
7	'Botón secundario'	4.5	7.2	'sí'
8	'Título del encabezado'	4.5	6.44	'sí'
9	'Subtítulo del encabezado'	4.5	5.09	'sí'
10	'Botón de peligro'	4.5	7.28	'sí'
11	'Texto de peligro en superficie'	4.5	7.28	'sí'
12	'Error sobre fondo'	4.5	6.14	'sí'
13	'Mensaje breve'	4.5	14.22	'sí'
14	'Icono primario sobre tarjeta'	3	6.15	'sí'
15	'Icono de peligro sobre tarjeta'	3	6.95	'sí'
16	'Pestaña activa'	3	6.44	'sí'
17	'Borde de campo de texto'	3	3.43	'sí'
18	'Barra de progreso en encabezado'	3	3.15	'sí'

Figura 1. Revisión de contraste del tema Verde (green).

```
ale@LAPTOP-7A9LFVMT UCRT64 ~/Documents/IPSS/todolist (master)
$ npm run check:contrast

> todolist@1.0.0 check:contrast
> node --disable-warning=MODULE_TYPELESS_PACKAGE_JSON scripts/check-contrast.ts

Tema: light (menor contraste de texto: 5.03:1, Filtro sin seleccionar)
```

(index)	pair	minimum	worst	ok
0	'Texto sobre fondo'	4.5	14.47	'sí'
1	'Texto sobre tarjeta'	4.5	16.41	'sí'
2	'Texto secundario sobre fondo'	4.5	5.76	'sí'
3	'Texto secundario sobre tarjeta'	4.5	6.53	'sí'
4	'Texto secundario sobre superficie'	4.5	6.85	'sí'
5	'Filtro sin seleccionar'	4.5	5.03	'sí'
6	'Filtro seleccionado y botón'	4.5	14.71	'sí'
7	'Botón secundario'	4.5	10.42	'sí'
8	'Título del encabezado'	4.5	11.84	'sí'
9	'Subtítulo del encabezado'	4.5	8.25	'sí'
10	'Botón de peligro'	4.5	7.28	'sí'
11	'Texto de peligro en superficie'	4.5	7.28	'sí'
12	'Error sobre fondo'	4.5	6.12	'sí'
13	'Mensaje breve'	4.5	15.88	'sí'
14	'Icono primario sobre tarjeta'	3	14.03	'sí'
15	'Icono de peligro sobre tarjeta'	3	6.94	'sí'
16	'Pestaña activa'	3	14.71	'sí'
17	'Borde de campo de texto'	3	3.16	'sí'
18	'Barra de progreso en encabezado'	3	4.52	'sí'

Figura 2. Revisión de contraste del tema Claro (light).

6. Componentes de interfaz

Tipo	Componente	Buenas prácticas aplicadas
Botones	Button, IconButton, botón flotante	Variantes por importancia, ripple en Android, atenuación en iOS y etiquetas para lector de pantalla.
Menús	ActionMenu	Orden, completar todas y eliminar completadas. Opciones no disponibles deshabilitadas.
Formularios	TextField, task-form	Validación con mensaje e icono, contador, foco en el primer error y tecla “siguiente”.
Listas	FlatList, TaskItem	Virtualización, identificador estable y estados vacíos con orientación.
Navegadores	Stack y Tabs	Tres pestañas con icono y texto, e insignia con las tareas pendientes.
Selección	ThemeOption	Grupo de opciones con muestra de color y estado seleccionado para lector de pantalla.
Diálogos	ConfirmDialog	Confirma acciones destructivas; funciona igual en todas las plataformas.
Mensajes	Snackbar	Confirma cada acción y se anuncia al lector de pantalla.

6.1. Estándares de Android e iOS

Android (Material Design)	iOS (Human Interface Guidelines)
Áreas táctiles de 48 dp	Áreas táctiles sobre el mínimo de 44 pt
Efecto ripple al tocar, del color del tema	Atenuación al tocar
Iconos Material Symbols	Iconos SF Symbols
Botón físico “atrás” controlado	Gesto de volver bloqueado si hay cambios sin guardar
Icono adaptable y monocromo	Icono de 1024 × 1024 sin transparencia
Respeta las barras del sistema	Respeta la isla o muesca y el indicador de inicio

Estas decisiones siguen las guías de diseño de cada plataforma. Cabe señalar que Google y Apple revisan cada aplicación al publicarla, por lo que seguirlas no garantiza su aprobación.

7. Usabilidad

Cada decisión responde a una necesidad concreta de quienes usan la aplicación:

Necesidad	Solución
Saber qué hacer al abrir la app por primera vez	Estado vacío con explicación y botón “Agregar tarea”
Agregar varias tareas seguidas	Botón “Guardar y agregar otra”, con el foco de vuelta en el título
Corregir un formulario incompleto	Mensajes específicos junto a cada campo y foco en el primer error
No perder lo escrito por accidente	Aviso antes de descartar, incluso con el botón “atrás” de Android
No eliminar por error	Diálogo de confirmación que nombra la tarea
Saber si una acción funcionó	Mensaje breve tras agregar, guardar o eliminar
Ver cuánto falta	Barra de progreso, porcentaje e insignia de pendientes
Conservar las tareas	Guardado automático en el dispositivo
Usar la app con lector de pantalla	Roles, estados y etiquetas descriptivas en los controles
Adaptar la app a sus gustos o a poca luz	Cinco temas elegibles y modo automático, con la elección guardada

8. Proceso de desarrollo

8.1. Preparación del entorno

```
node --version                        # v24.22.2
npx create-expo-app todolist --template default@sdk-58
npx expo install @react-native-async-storage/async-storage
npx expo install eslint eslint-config-expo -- --dev
npx expo install culori @types/culori playwright-core -- --dev
npm run web                           # http://localhost:8100
```

8.2. Estructura de archivos

```
src/
├─ app/                    Rutas (cada archivo es una pantalla)
│  ├─ _layout.tsx          Providers y navegador Stack
│  ├─ task-form.tsx        Formulario de crear y editar
│  └─ (tabs)/
│     ├─ _layout.tsx       Navegador de pestañas
│     ├─ index.tsx         Lista de tareas
│     ├─ summary.tsx       Resumen de avance
│     └─ settings.tsx      Ajustes y elección de tema
├─ components/             Componentes de la aplicación
│  ├─ empty-state.tsx, filter-tabs.tsx, screen-container.tsx
│  ├─ screen-header.tsx, task-item.tsx, theme-option.tsx
│  ├─ themed-text.tsx
│  └─ ui/                  Componentes genéricos reutilizables
│     ├─ action-menu.tsx, button.tsx, confirm-dialog.tsx
│     └─ icon.tsx, icon-button.tsx, progress-bar.tsx, text-field.tsx
├─ constants/              Medidas (theme.ts) y colores (themes.ts)
├─ context/                Estado global (tema, tareas y mensajes)
├─ hooks/                  Tema y esquema de color
├─ services/               Almacenamiento de tareas y tema
├─ types/                  Tipos de datos
└─ utils/task-utils.ts     Validación, filtros y estadísticas

scripts/                    Revisión de contraste y prueba de flujos
```

8.3. Verificación

Para comprobar el funcionamiento de la aplicación, ejecutamos las siguientes revisiones. La prueba automatizada usa la librería Playwright para recorrer la aplicación en el navegador, con tamaño de teléfono, y la complementamos con una prueba manual. Las Figuras 3 a 7 muestran los resultados.

Prueba	Resultado
Tipos con tsc --noEmit	Sin errores
Lint con expo lint	Sin errores

Prueba	Resultado
Contraste con npm run check:contrast	95 de 95 combinaciones
Prueba automatizada de flujos en navegador con npm run test:flows	33 de 33 comprobaciones, sin errores en consola
Configuración con npx expo-doctor	20 de 20 comprobaciones
Compilación para web, Android e iOS con npx expo export	Correcta
Prueba manual de todos los flujos (agregar, editar, eliminar, filtros y temas) en navegador y teléfono	Sin errores

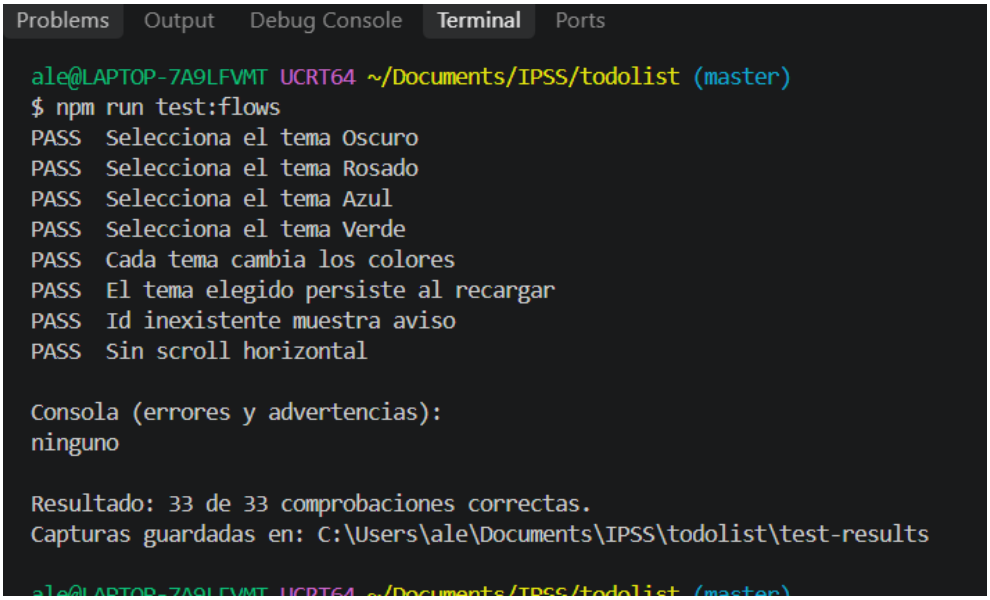


Figura 3. Prueba de flujos en navegador: 33 de 33 comprobaciones.

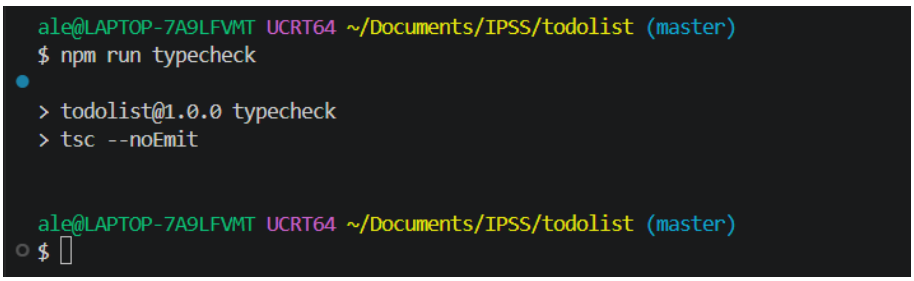


Figura 4. Revisión de tipos con TypeScript, sin errores.

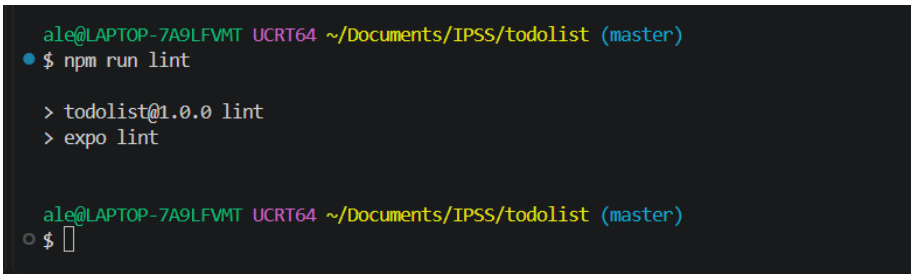


Figura 5. Revisión de código con ESLint, sin errores.

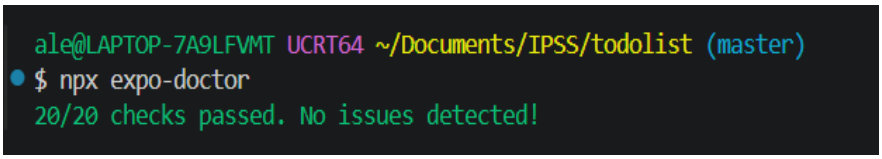


Figura 6. Revisión de configuración con expo-doctor.



Figura 7. Compilación para web, Android e iOS.

8.4. Ejecución y repositorio

Para ejecutar el proyecto, se instalan las dependencias con npm install y se inicia con npm start. Luego, la aplicación se abre en un teléfono con Expo Go, escaneando el código QR, o en el navegador con npm run web.

Repositorio: https://github.com/AndreaVCC/EVU1_TODOLIST.git

9. Capturas de pantalla



Figura 8. Estado inicial vacío



Figura 9. Lista con una tarea completada

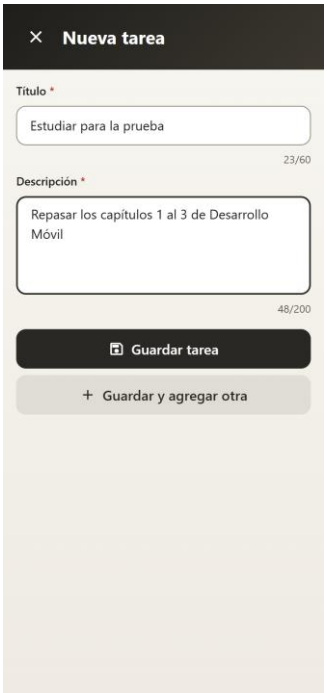


Figura 10. Formulario de nueva tarea

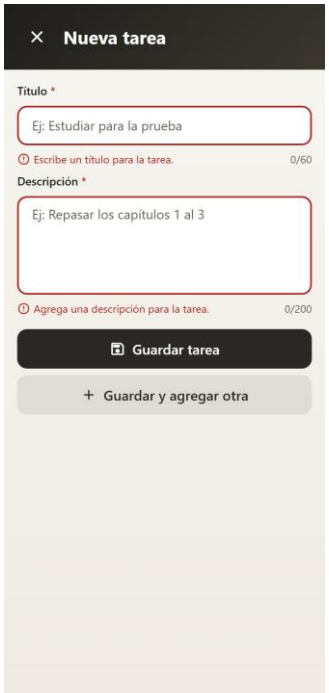


Figura 11. Validación de campos



Figura 12. Edición de una tarea



Figura 13. Aviso de cambios sin guardar



Figura 14. Confirmación al eliminar

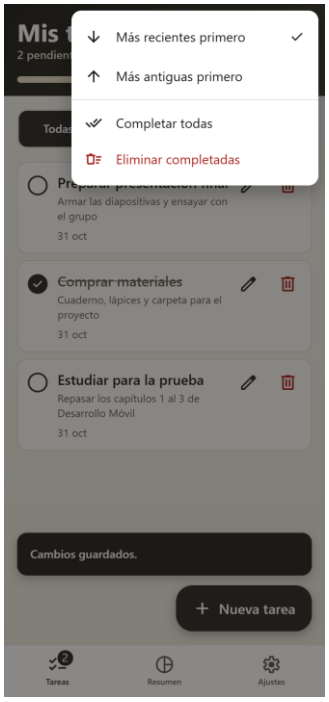


Figura 15. Menú de opciones

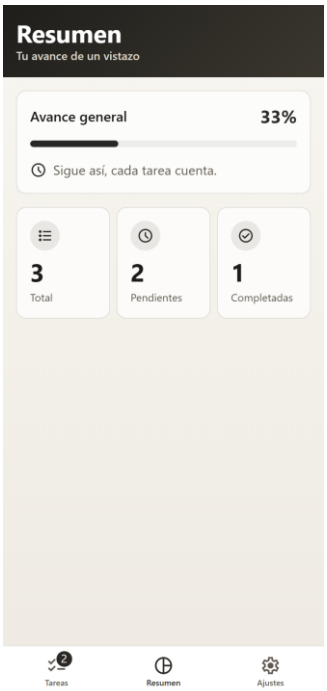


Figura 16. Pantalla de resumen

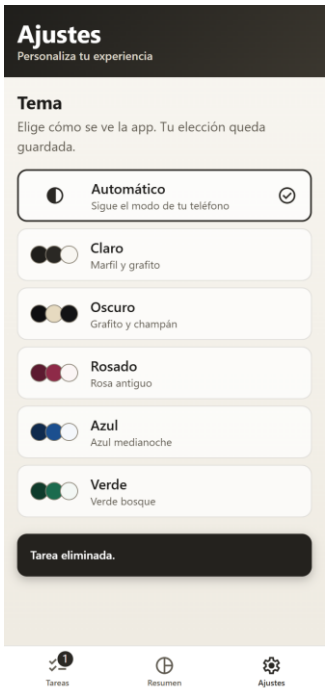


Figura 17. Ajustes: elección de tema



Figura 18. Tema Oscuro



Figura 19. Tema Rosado



Figura 20. Tema Azul



Figura 21. Tema Verde

10. Evolución del proyecto

Dejamos la estructura preparada para las funcionalidades de las próximas unidades:

- **Almacenamiento local:** ya está implementado y podría migrarse a una base de datos modificando únicamente la capa de servicio.
- **Autenticación:** se puede incorporar con un nuevo Provider y un grupo de rutas protegidas.
- **APIs externas:** la capa de servicio puede sincronizar las tareas con un servidor, sin cambiar las pantallas.

11. Conclusiones

Desarrollamos una aplicación que cumple los requisitos de la pauta: permite agregar, visualizar, editar y eliminar tareas con título y descripción, y limpia el formulario después de agregar una tarea. Separar la interfaz, el estado y los datos nos dejó un código ordenado y fácil de ampliar en las próximas unidades. A su vez, el sistema de diseño común dio consistencia visual a la aplicación, y las decisiones de usabilidad ayudan a evitar errores y a orientar a las personas en cada paso.

12. Referencias

- Expo. (2026). Expo SDK 58 changelog. https://expo.dev/changelog/sdk-58
- Expo. (2026). Expo Router: Stack, Tabs y migración de SDK 57 a 58. https://docs.expo.dev/router/introduction/
- Expo. (2026). Symbols (expo-symbols). https://docs.expo.dev/versions/v58.0.0/sdk/symbols/
- Meta. React Native: About the New Architecture. https://reactnative.dev/architecture/landing-page
- W3C. (2023). Web Content Accessibility Guidelines (WCAG) 2.2. https://www.w3.org/TR/WCAG22/
- OpenJS Foundation. Node.js. https://nodejs.org/
- Evercoder. Culori: a comprehensive color library for JavaScript. https://culorijs.org/
- Google. Material Design 3. https://m3.material.io/
- Apple. Human Interface Guidelines. https://developer.apple.com/design/human-interface-guidelines/